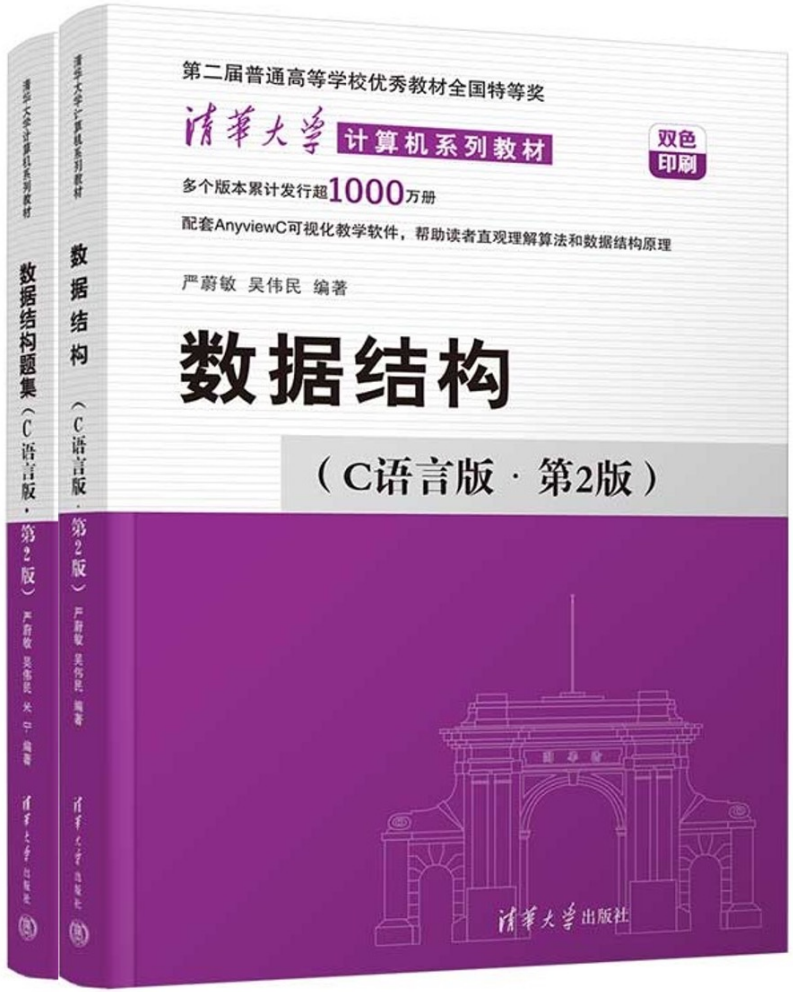


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第12章

文件和索引结构

12.1 文件基础

12.1.1 文件的基本概念

1. 文件及其类别

文件(file)是由大量性质相同的记录组成的集合。可按其记录的类型不同而分成两类：操作系统的文件和数据库文件。

操作系统中的文件仅是一维的连续的字符序列，无结构、无解释。它也是记录的集合，这个记录仅是一个字符组，用户为了存取、加工方便，把文件中的信息划分成若干组，每一组信息称为一个逻辑记录，且可按顺序编号。

数据库中的文件是带有结构的记录的集合；这类记录是由一个或多个数据项组成的集合，它也是文件中可存取的数据的基本单位。

2. 记录的逻辑结构和物理结构

记录的逻辑结构是指记录在用户或应用程序员面前呈现的方式，是用户对数据的表示和存取方式。

记录的物理结构是数据在物理存储器上存储的方式，是数据的物理表示和组织。

在物理记录和逻辑记录之间可能存在下列3种关系：

(1) 一个物理记录存放一个逻辑记录。

(2) 一个物理记录包含多个逻辑记录。

(3) 多个物理记录表示一个逻辑记录。

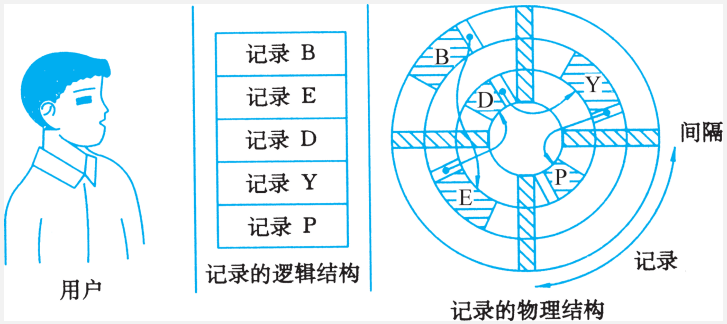


图12.2　记录的逻辑结构与物理结构差别示例

12.1.1 文件的基本概念(续)

3. 文件的操作(运算)

文件的操作有两类：检索和修改。

文件的检索有下列3种方式：

(1) 顺序存取：存取下一个逻辑记录。

(2) 直接存取：存取第i个逻辑记录。

以上两种存取方式都是根据记录序号(即记录存入文件时的顺序编号)或记录的相对位置进行存取的。

(3) 按关键字存取：给定一个值，查询一个或一批关键字与给定值相关的记录。对数据库文件可以有如下4种查询方式：

① 简单询问：查询关键字等于给定值的记录。

例如，在图12.1的文件中，给定一个准考证号码或学生姓名，查询相关记录。

② 区域询问：查询关键字属某个区域内的记录。

例如，在图12.1的文件中查询某某中学的学生成绩，则给定准考证号的某个数值范围。

③ 函数询问：给定关键字的某个函数。

例如查询总分在全体学生的平均分以上的记录或处于中值的记录。

④ 布尔询问：以上3种询问用布尔运算组合起来的询问。

例如，查询总分600分以上且数学100分以上，或者总分在平均分以下的外语98分以上的全部记录。

文件的修改包括插入一个记录、删除一个记录和更新一个记录3种操作。

12.1.2 顺序文件

顺序文件(Sequential File)是记录按其在文件中的逻辑顺序依次进入存储介质而建立的，即顺序文件中物理记录的顺序和逻辑记录的顺序是一致的。若次序相继的两个物理记录在存储介质上的存储位置是相邻的，则又称连续文件；若物理记录之间的次序由指针相链表示，则称串联文件。

顺序文件是根据记录的序号或记录的相对位置来进行存取的文件组织方式。它的特点是：

(1) 存取第i个记录，必须先搜索在它之前的$i-1$个记录。

(2) 插入新的记录时只能加在文件的末尾。

(3) 若要更新文件中的某个记录，则必须将整个文件进行复制。

由于顺序文件的优点是连续存取的速度快，因此主要用于只进行顺序存取、批量修改的情况。若对应答时间要求不严时亦可进行直接存取。

对顺序文件进行顺序查找类似于第9章讨论的顺序查找，其平均查找长度为$(n'+1)/2$，其中n'为文件所含物理记录的数目(相对外存读/写而言，内存查找的时间可以忽略不计)。对磁盘文件可以进行分块查找或折半查找(对不定长文件不能进行折半查找)。但是，若文件很大，在磁盘上占多个柱面时，折半查找将引起磁头来回移动，增加寻查时间。

假若某个顺序文件，其记录修改的频率较低，则用批处理并不适宜，此时可另建立一个附加文件，以存储新插入和更新后的记录，待附加文件增大到一定程度时再进行批处理。在检索时可以先查主文件，若不成功再查附加文件，或反之。显然这将增加检索的时间，但可以采取其他措施弥补之，详细情况可参阅参考书目[13]。

算法12.1 账目文件成批更新处理

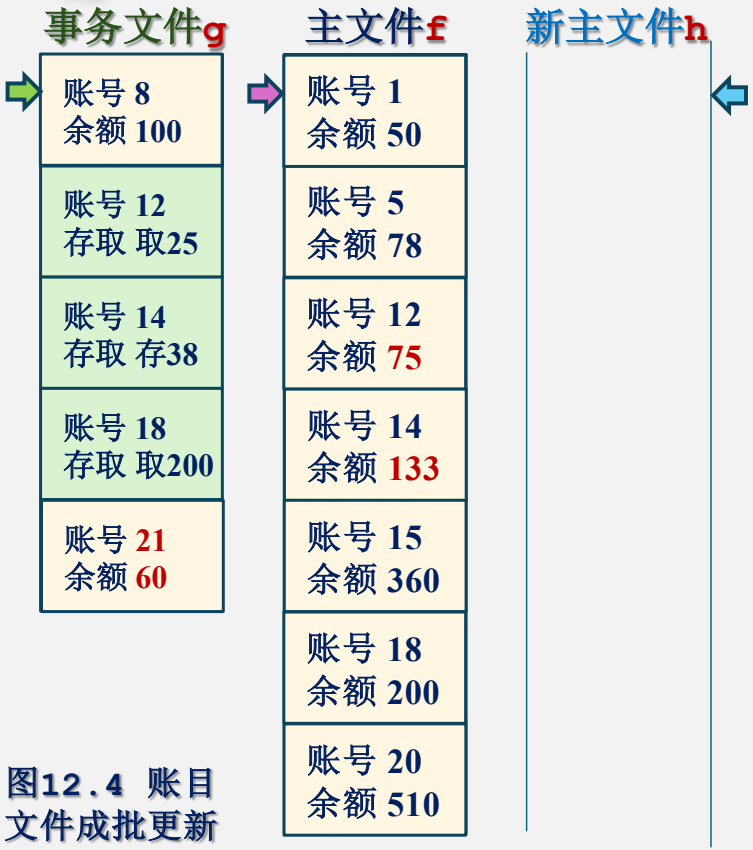


图12.4 账目文件成批更新

随着计算机与网络技术的飞速发展，银行及各种线上业务已基本实现实时更新。但在现实社会中，算法12.1实现的顺序文件的批量更新还有一定的用途，比如仓储的定期盘点，实物与账本会有偏差。那么，就可以根据盘点的实物数据形成的事务文件，实现对主文件批量更新。

第12章已到了课程的末尾，对算法12.1的学习，还可以回顾相关的一系列算法，如：

算法2.1 —— 集合求并集

算法2.2、2.7和2.21 —— 归并线性表、顺序表和单链表；

算法2.23 —— 稀疏多项式相加

算法4.5、4.6 —— 串的模式匹配

算法5.3 —— 稀疏矩阵相乘

这些设计两个线性结构或矩阵的处理算法，从简单到复杂。通过对比复习归纳，可以得到提高。

算法12.1的主体就是一个while循环，从事务文件和主文件顺序读入记录，对比账号：

账号小的，复制写入新主文件；

账号相同的，更新后写入新主文件。

请看图12.4所示例子的更新处理过程演示。

12.1.3 索引文件

除了文件本身(称做数据区)之外，另建立一张指示逻辑记录和物理记录之间一一对应关系的表——索引表。这类包括文件数据区和索引表两大部分的文件称做**索引文件**。

图12.5为两个简单的索引表示例。索引表中的每一项称做**索引项**。不论主文件是否按关键字有序，索引表中的索引项总是按关键字(或逻辑记录号)顺序排列。若数据区中的记录也按关键字顺序排列，则称**索引顺序**文件。反之，若数据区中记录不按关键字顺序排列，则称**索引非顺序**文件。

逻辑记录号	标识①	物理记录号
0	1	4
1	1	7
2	0	
3	1	10

关键字 k_i	物理记录号
101	15
119	04
123	31
125	11

图12.5　索引表示例

索引表随数据存储的变化而动态生成和维护。在记录输入建立数据区的同时建立一个索引表，表中的索引项按记录输入的先后次序排列，待全部记录输入完毕后再对索引表进行排序。

索引文件的检索方式为直接存取或按关键字(进行简单询问)存取，检索过程和第9章讨论的分块查找相类似

删除或更新一个记录时，须同步维护索引项。

当记录数目很大时，可建立多级索引。

当数据文件在使用过程中记录变动较多时，应采用动态索引。如二叉排序树(或二叉平衡树)、B树以及键树，这些都是树表结构，插入、删除都很方便。又由于它本身是层次结构，则无需建立多级索引，而且建立索引表的过程即排序的过程。

12.2 索引顺序文件

12.2.1 索引顺序文件概述

顺序索引文件（Indexed Sequential File）是一种结合了**索引结构**和**顺序存储**的混合文件组织方式，旨在平衡**随机访问**和**范围查询**的性能。其核心设计如下：

- **顺序存储**。数据按逻辑顺序存储在连续的物理块（或页）中，块内记录按键排序，块间通过索引表管理。
- **索引结构**。维护一个索引表，每个索引项包含键（Key）和对应记录的物理位置（如文件偏移量）。索引项按键排序，支持高效查找（如二分查找），快速定位磁盘目标块。
- **适用场景**。适合需要高效范围查询（如时间范围统计）且兼顾随机访问（如单条记录查询）的场景。

例12-2 文件结构示例。假设一个文件存储用户订单信息，每个块大小为4条记录，索引表存储块起始键和块偏移：
其中，**数据块0**（偏移0）：记录键为100、150、180、200。
数据块1（偏移4096）：记录键为200、250、280、300。

索引项	起始键	块偏移
100	100	0
200	200	4096
300	300	8192

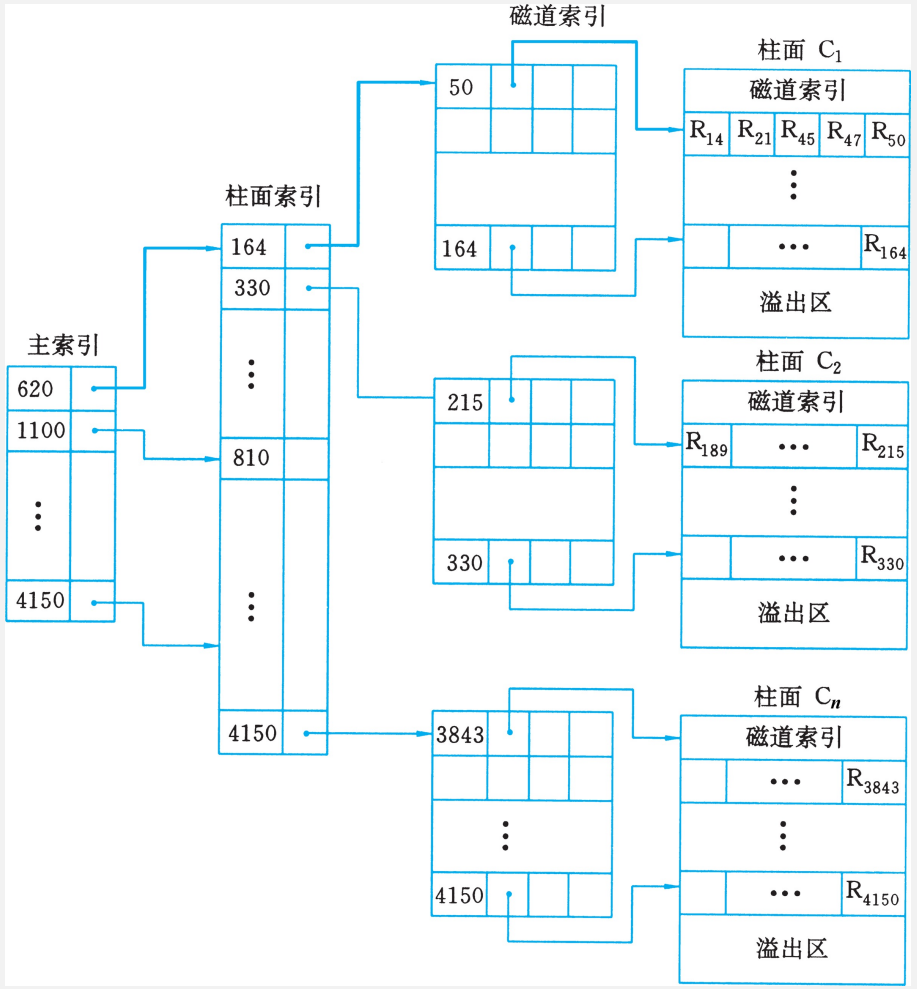
清華大學出版社 TSINGHUA UNIVERSITY PRESS

顺序索引文件的核心操作及性能。

操作	实现逻辑	时间复杂度	适用性
插入	1. 定位插入块（通过索引表）。 2. 插入记录并保持块内有序。 3. 若块满，分裂为两个块并更新索引。	O(log m) + O(k)	中等（需块分裂开销）
查找	1. 通过索引表二分查找目标块。 2. 在块内顺序查找记录。	O(log m) + O(k)	高效（块内顺序扫描优化）
范围查询	1. 定位起始块和结束块。 2. 顺序读取所有块内的符合条件的记录。	O(log m) + O(n)	最优（块连续存储）
删除	1. 定位记录所在块。 2. 标记记录为无效（惰性删除）。 3. 定期整理块以回收空间。	O(log m) + O(k)	低效（需后续整理）

12.2.2 ISAM文件

索引顺序存取方法ISAM为Indexed Sequential Access Methed 的缩写，它是一种专为磁盘存取设计的文件组织方式。由于磁盘是以盘组、柱面和磁道三级地址存取的设备，则可对磁盘上的数据文件建立盘组、柱面和磁道这里的磁道索引，实际为盘面索引，为遵循习惯仍称磁道索引。三级索引。文件的记录在同一盘组上存放时，应先集中放在一个柱面上，然后再顺序存放在相邻的柱面上，对同一柱面，则应按盘面的次序顺序存放。



在ISAM文件上检索记录时，先从主索引出发找到相应的柱面索引，再从柱面索引找到记录所在柱面的磁道索引，最后从磁道索引找到记录所在磁道的第一个记录的位置，由此出发在该磁道上进行顺序查找直至找到为止；反之，若找遍该磁道而不存在此记录，则表明该文件中无此记录。例如，查找关键字为21的记录时的查找路径如图12.8中的粗实线所示。

12.3 B+树索引文件

12.3.1 VSAM文件

虚拟存储存取方法VSAM是Virtual Storage Access Method的缩写。这种存取方法利用了操作系统的虚拟存储器的功能，给用户提供方便。

顺序集和索引集一起构成一棵**B+树**，为文件的索引部分。

VSAM文件既可在顺序集中进行顺序存取，又可从最高层的索引(B+树的根节点)出发进行按关键字存取。顺序集中一个节点连同其对应的所有控制区间形成一个整体，称做**控制区域**(Control Range)。每个控制区间可视为一个逻辑磁道，而每个控制区域可视为一个逻辑柱面。

VSAM文件中没有**溢出区**，解决插入的办法是在初建文件时留有空间。

VSAM文件的结构如图12.11所示。它由3部分组成：索引集、顺序集和数据集。文件的记录均存放在数据集中，数据集中的一个节点称为**控制区间**(Control Interval)，它是一个I/O操作的基本单位，它由一组连续的存储单元组成。控制区间的大小可随文件不同而不同，但同一文件上控制区间的大小相同。每个控制区间含有一个或多个按关键字递增有序排列的记录。

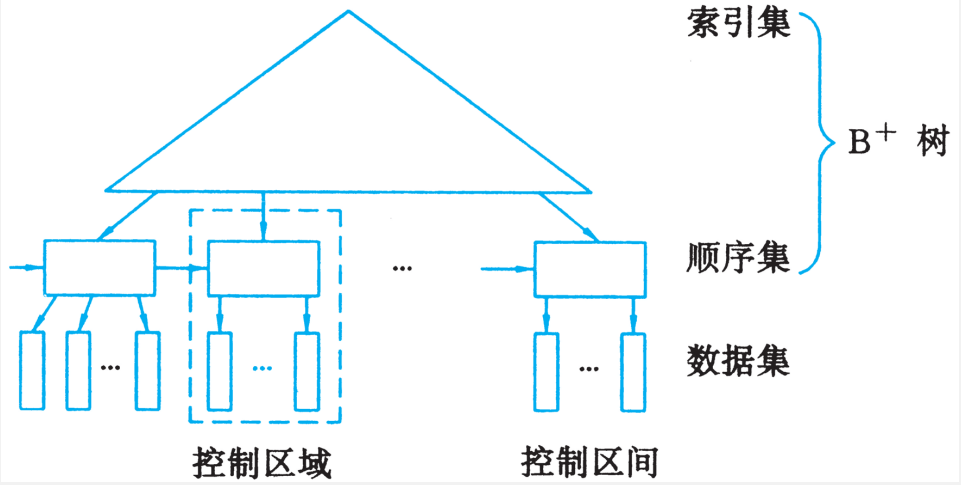


图12.11 VSAM文件的结构示意图

12.3.2 现代B+树索引存储方式对VSAM的改进

1. 现代B+树索引存储方式对早期VSAM的改进

（1）动态索引结构的优化。VSAM的B+树是静态的，节点分裂和合并需要预定义规则，难以适应动态数据变化。

（2）存储介质适配性增强。VSAM的磁盘优化是针对针对传统机械硬盘（HDD）的寻道时间优化，但对SSD的随机访问特性适配不足。

（3）其他改进还包括：并发控制与事务支持、数据压缩与存储效率、事务与崩溃恢复、混合存储与分层索引、缓存与预读取策略、以及分布式与扩展性等等。

这些都是后续关于数据库系统及设计的专业课程将涉及的内容。

2. 实际应用场景

- **VSAM的遗留场景：**仍用于IBM大型主机系统（如银行核心交易系统），但逐步被现代数据库替代。
- **现代B+树的应用：**
  - **联机事务处理（OLTP）系统：**MySQL InnoDB支持高并发事务。
  - **分析型数据库：**ClickHouse的MergeTree引擎结合B+树与列式存储。
  - **分布式系统：**Cassandra的Partition Key索引支持海量数据分片。

现代B+树索引通过**动态结构、SSD优化、并发控制、混合存储**等改进，解决了VSAM的局限性，并在性能、扩展性和功能上实现了全面超越。其核心目标始终是**平衡随机访问与顺序扫描的效率**，同时适应不断变化的存储硬件和应用需求。

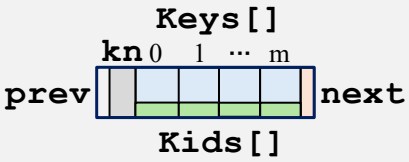
12.3.3 B+树的实现

可在AnyviewC可视交互跟踪算法过程中B+树的形态变化

1. B+树的存储结构和初始化

```
typedef struct Node {
    int       kn;           // 节点当前键数
    NodeKind  kind;         // 节点种类
    KeyType   keys[M+1];    // 键数组，M号单元非必要，但方便插入及分裂等操作
    Node     *kids[M+1];    // 孩子分支指针数组，M号单元也为操作便利
    Node     *prev, *next;  // 叶子节点有序双向链指针
} Node, *BpTree;            // 节点类型，B+树指针类型
```

m阶B+树的结点图示：



不低于3阶的B+树的一种初始化约定

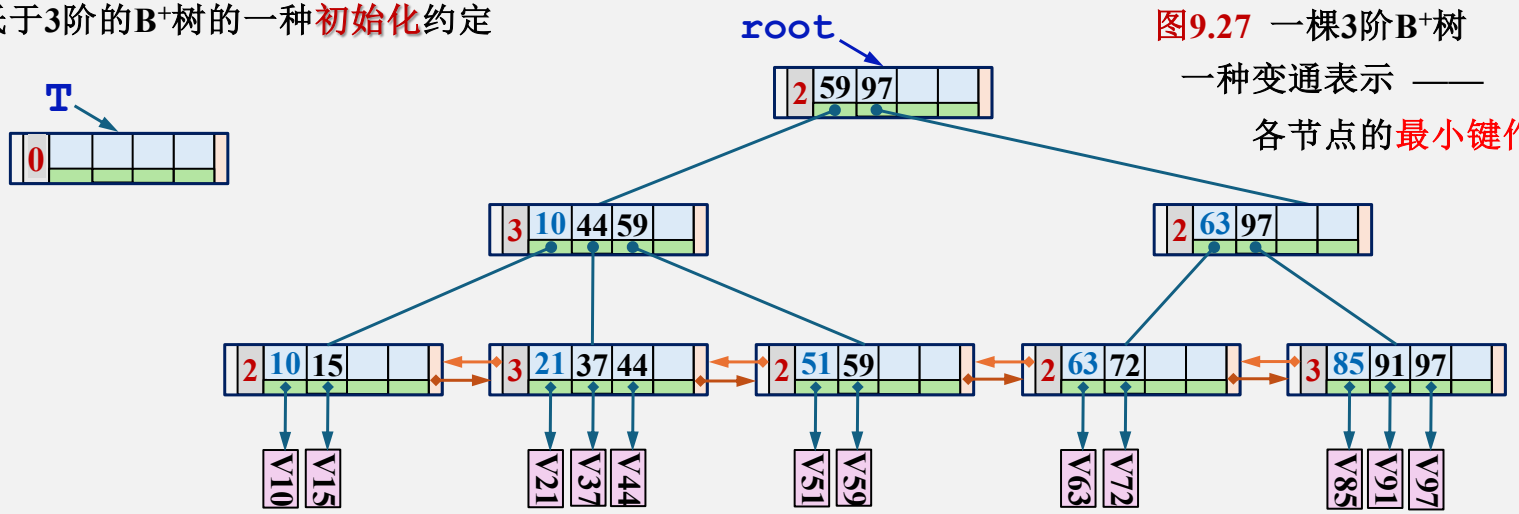


图9.27 一棵3阶B+树

一种变通表示 —— 各节点的最小键作为索引

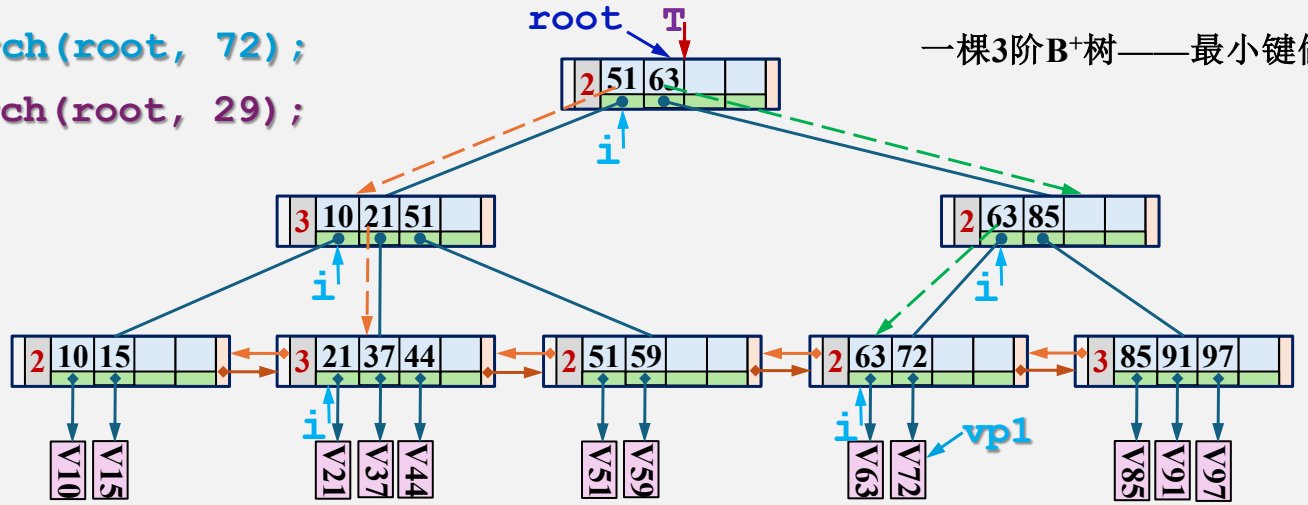
12.3.3 B+树的实现

2. 查找

root 29

```
ValueType Search(BpTree T, int key) {    // 算法 12.2，查找并返回键key的值
  int idx;
  while (T!=NULL) {
    for (idx=0; idx<T->kn && key>=T->keys[idx]; idx++);  // 在*T内查找分支
    if (idx != 0) idx--;                                 // 修正idx值
    if (T->kind==LEAF)                                   // 若是叶子
      if (idx<T->kn && T->keys[idx]==key) return (char*)T->kids[idx]; // 找到
      else return NULL;                                                // 不存在
    T = T->kids[idx];                                    // 沿idx分支继续向下
  }
  return NULL;                                           // 未找到，返回NULL
}
```

```
vp1 = Search(root, 72);
vp2 = Search(root, 29);
```



一棵3阶B+树——最小键做索引

清華大學出版社 TSINGHUA UNIVERSITY PRESS

3. 插入 —— 对插入算法的简约表达（为了后续讨论简便）

```
BpTree Insert_rec(BpTree T, KeyType key, ValueType value, int i, BpTree parent) {
  int j;                                  // 在*parent的第i子树实施递归，将键值对插入到恰当的叶子
  for (j=0; j < T->kn && key >= T->keys[j];  j++)          // *T内查找分支
    if (key==T->keys[j]) return T;                         // 若已存在，则不重复插入
  if (j != 0 && T->kind==INTERNAL) j--;                    // 修正j值
  if (T->kind==LEAF)                                       // 若是叶子
    T = InsertKey(                                         // 则插入键值对
  else                                                     // 否则是内部节点
    T->kids[j] = Insert_rec(T->kids[j], key, value, j, T); // 沿j分支递归
  if (T->kn > M) {                                         // 若节点超大，则调整
    if (parent==NU                                         // 若是根，则分裂节点
    else {                                                 // 否则
      Node* bro =                                          // 若寻找到未满的兄弟
      if (bro!=NUL                                         // 则将*T一个元素迁移给它
      else T = SplitNode(parent, T, i);                    // 未找到，则分裂*T
    }
  }
  if (parent!=NULL) parent->keys[i] = T                    [0];  // 刷新*T的索引
  return T;                                                // 返回T(可能已被更改)
}

BpTree Insert(BpTr                                         插入主函数
  { return Insert_                                         归实施插入键值对，根无父节点
```

在T节点确定向下分支号j，若key已存在，则结束；

若T节点是叶子，则
  Insertkey(parent,T,(void*)value,i,j);
否则递归
  Insert_rec(T->kids[j],key,value,j,T);

若T节点超大，则调整：
  若T节点是根，则分裂 T=SplitNode(parent);
  否则寻找未满兄弟

刷新T节点在父节点的索引后，return T;

若 bro!=NULL,
则 MoveElem(T,bro,parent,i,1);

12.3.3 B+树的实现

3. 插入

递归插入算法

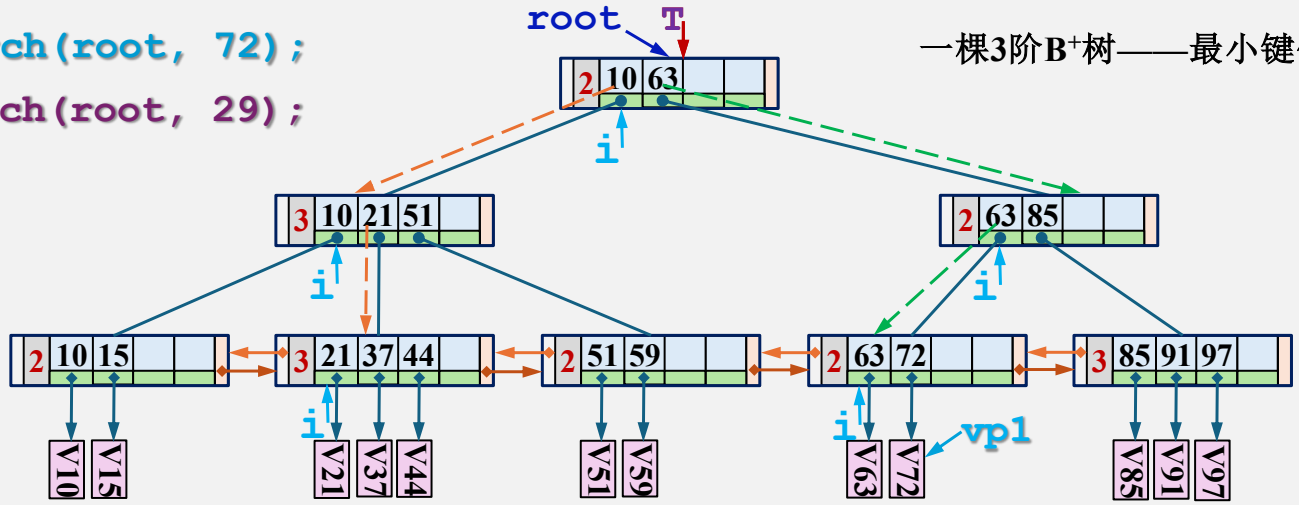
在T节点确定向下分支号j，若key已存在，则结束；

若T节点是叶子，则
```
Insertkey(parent,T,(void*)value,i,j);
```
否则递归
```
Insert_rec(T->kids[j],key,value,j,T);
```

若T节点超大，则调整：
若T节点是根，则分裂 `T=SplitNode(parent);`
否则寻找未满兄弟
```
bro=FindSiblingUnfull(parent,i);
```
若 `bro!=NULL`，
则 `MoveElem(T,bro,parent,i,1);`
否则分裂 `T=SplitNode(parent,T,i);`

刷新T节点在父节点的索引后，`return T;`

```
vp1 = Search(root, 72);
vp2 = Search(root, 29);
```



一棵3阶B+树——最小键做索引

12.3.3 B+树的实现

3. 插入　示例演示一

推荐在AnyviewC可视交互跟踪算法过程

```
递 2  Kids[1],33,v33,1,Kids[0]
归 1  Kids[0],33,v33,0,root
栈 0     root,33,v33,0,NULL
```

形参：T, key,value,i,parent

插入键值对：33 – v33

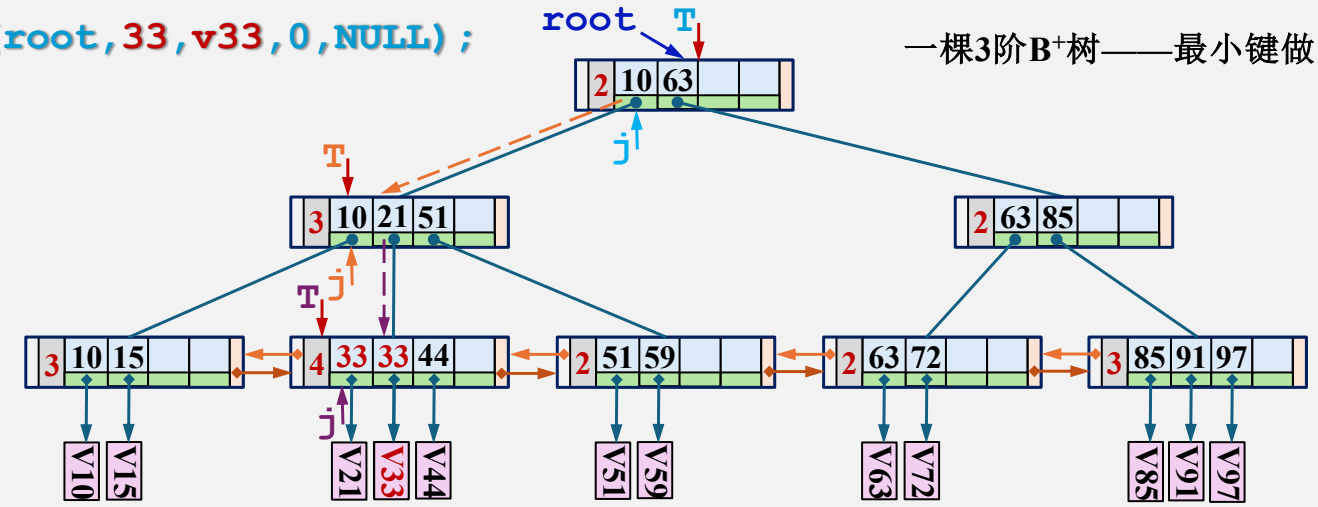
```
root=Insert_rec(root,33,v33,0,NULL);
```

递归插入算法

```
在T节点确定向下分支号j，若key已存在，则结束；
若T节点是叶子，则
   Insertkey(parent,T,(void*)value,i,j);
否则递归
   Insert_rec(T->kids[j],key,value,j,T);
若T节点超大，则调整：
   若T节点是根，则分裂 T=SplitNode(parent);
   否则寻找未满兄弟
      bro=FindSiblingUnfull(parent,i);
      若 bro!=NULL,
      则 MoveElem(T,bro,parent,i,1);
      否则分裂 T=SplitNode(parent,T,i);
刷新T节点在父节点的索引，return T;
```

InsertKey

MoveElem_aux



一棵3阶B+树——最小键做索引

12.3.3 B+树的实现

3. 插入　示例演示二

推荐在AnyviewC可视交互跟踪算法过程

递归栈
2 Kids[0],8,v08,0,Kids[0]
1 Kids[0],8,v08,0,root
0 root,8,v08,0,NULL

形参：T, key,value,i,parent

递归插入算法

```
在T节点确定向下分支号j，若key已存在，则结束；
若T节点是叶子，则
   Insertkey(parent,T,(void*)value,i,j);
否则递归
   Insert_rec(T->kids[j],key,value,j,T);
若T节点超大，则调整：
   若T节点是根，则分裂 T=SplitNode(parent);
   否则寻找未满兄弟
      bro=FindSiblingUnfull(parent,i);
      若 bro!=NULL,
      则 MoveElem(T,bro,parent,i,1);
      否则分裂 T=SplitNode(parent,T,i);
刷新T节点在父节点的索引，return T;
```

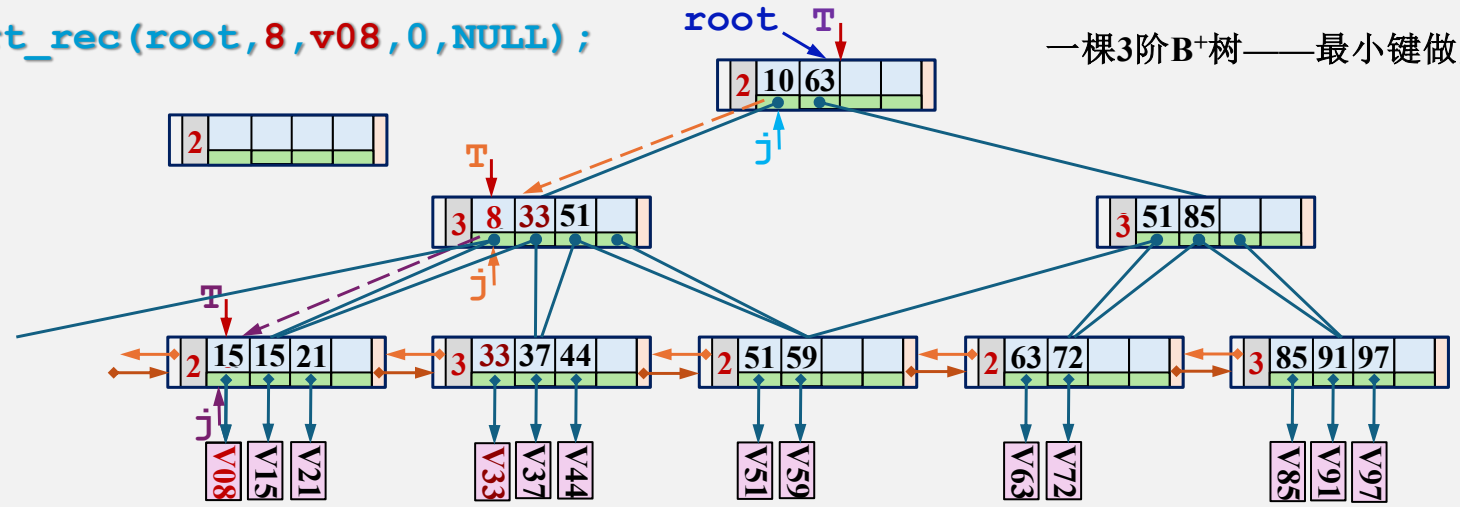
InsertKey

MoveElem_aux

插入键值对：8 – v08

```
root=Insert_rec(root,8,v08,0,NULL);
```

一棵3阶B+树——最小键做索引



12.3.3 B+树的实现

4. 删除　示例演示一（借键）

推荐在AnyviewC可视交互跟踪算法过程

递归栈
2 Kids[0],63,0,Kids[0]
1 Kids[1],63,1,root
0 root,63,0,NULL

形参：T, key,i,parent

递归删除算法

在T节点确定向下分支号j；

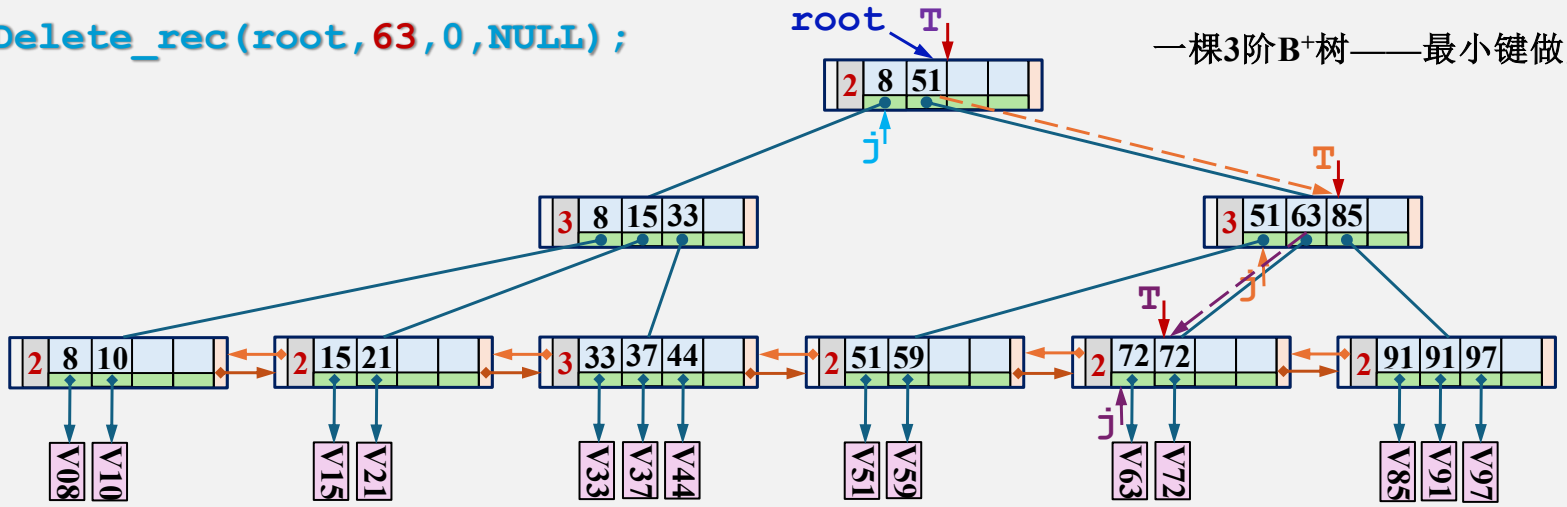
若T节点是叶子，若key不存在，则结束；
存在则 Deletekey(parent,T,i,j)；
否则递归 Delete_rec(T->kids[j],key,j,T)；

若*T是根且非叶子且键少于2 或 非根且键过少则调整
若*T是根，若是内部节点且键少于2，则用该孩(或NULL)顶替；
否则寻找富余兄弟
bro=FindSiblingRich(parent,i,&j)；
若 bro!=NULL，则 MoveElem(bro,T,parent,j,1)；
否则寻找合适的左或右兄弟*bro，与之合并
parent=MergeNode(parent,T,bro,i)；
T=parent->kids[i]；（T指向其父的第i孩子）

刷新T节点在父节点的索引，return T；

删除键：63

```
root=Delete_rec(root,63,0,NULL);
```



一棵3阶B+树——最小键做索引

12.3.3 B+树的实现

4. 删除　示例演示二（合并）

推荐在AnyviewC可视交互跟踪算法过程

递归栈
2 Kids[0],59,0,Kids[0]
1 Kids[1],59,1,root
0 root,59,0,NULL

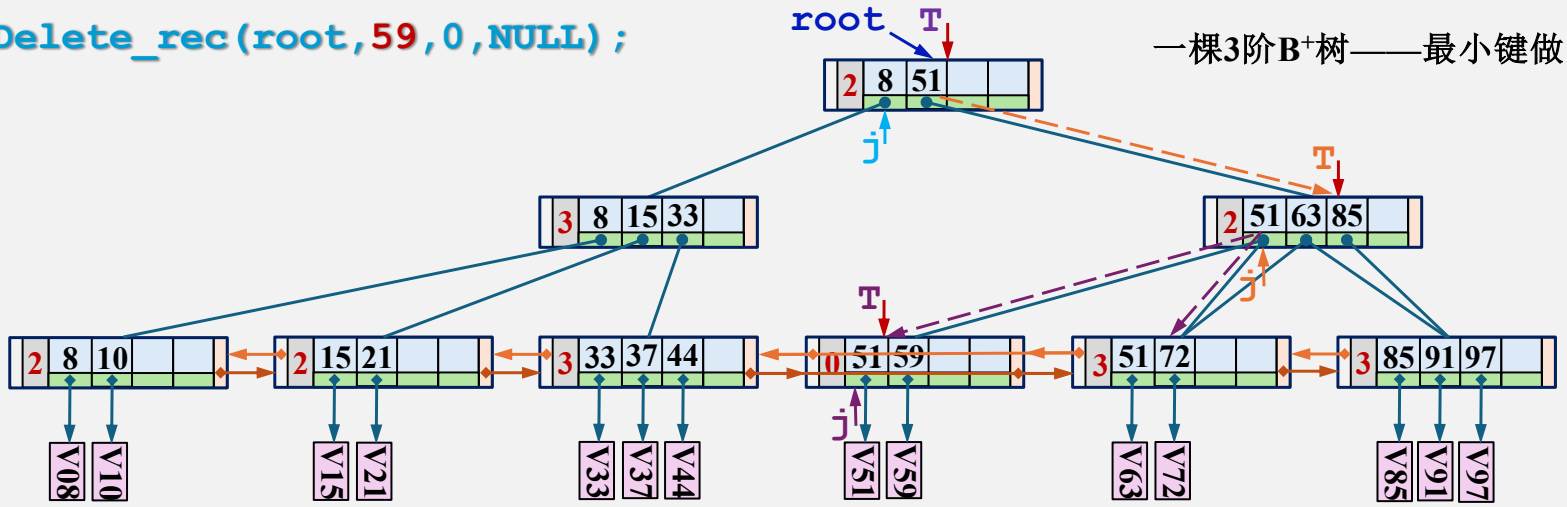
形参：T, key,i,parent

删除键：59

```
root=Delete_rec(root,59,0,NULL);
```

递归删除算法

```
在T节点确定向下分支号j;
若T节点是叶子，若key不存在，则结束;
                存在则 Deletekey(parent,T,i,j);
否则递归 Delete_rec(T->kids[j],key,j,T);
若*T是根且非叶子且键少于2 或 非根且键过少则调整
  若*T是根，若是内部节点且键少于2，则用该孩(或NULL)顶替;
否则寻找富余兄弟
    bro=FindSiblingRich(parent,i,&j);
  若 bro!=NULL，则 MoveElem(bro,T,parent,j,1);
  否则寻找合适的左或右兄弟bro，与之合并
    parent=MergeNode(parent,T,bro,i);
    T=parent->kids[i]; (T指向其父的第i孩子)
刷新T节点在父节点的索引，return T;
```



一棵3阶B+树——最小键做索引

12.5.3 全文检索的散列倒排索引示例

全文检索是倒排索引的一个典型应用场景。

```
typedef int DocIdxType;     // 整型表示文档内第几个
typedef struct DocIdxList {
   int doc_idx;             // 键的一个位置索引值
   DocIdxList *next;        // 位置索引链指针
} DocIdxList;               // 键的全文的位置索引表
typedef struct HashNode {
   char        *keyword;    // 键（字符串）
   DocIdxList *idxlist;     // 文档内位置索引表指针
   HashNode    *next;       // 同义词链指针
} HashNode;                 // 拉链散列表结点类型
typedef struct {
   HashNode  **list;    // 散列表向量
   int         size;    // 向量长度（散列地址区间长
} *ChainHT;             // 拉链散列表指针类型
ChainHT InitChainHT(int size) {  // 初建拉链散列表
   ChainHT ht;
   if (!(ht = (ChainHT)malloc(sizeof(*ht)))) exit(OVERFLOW);
   if (!(ht->list = (HashNode**)calloc(size, sizeof(HashNode*)))) exit(OVERFLOW);
   ht->size = size;      // 分配存储空间后，保留散列地址区间长度
   return ht;            // 返回初建的拉链散列表
}
```

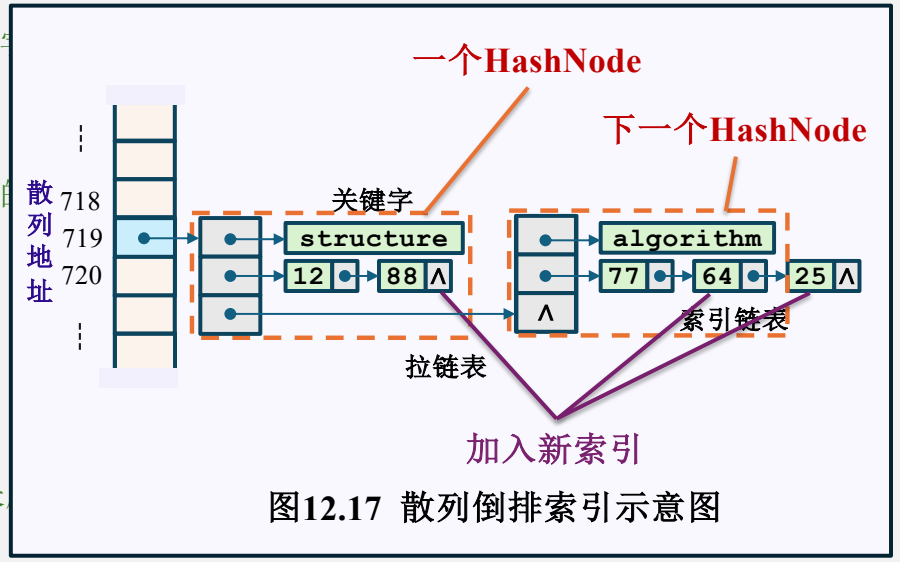


图12.17 散列倒排索引示意图